

THE ML / DL HANDBOOK — COMPANION SERIES

# Volume IV

# Unsupervised Learning

---

The Expanded Companion Deep-Dive to Part IV of the ML/DL Handbook — full derivations, worked numerical examples, and practitioner notes for PCA and clustering, the two classic answers to “what structure is hiding in this data?”

Chapter 12 — PCA, or Finding the Grain of the Wood

Chapter 13 — K-Means & Clustering, Finding the Natural Groups

July 16, 2026

## How to Read This Volume

*No labels here — just data, and the question “what structure is hiding in it?” Two classic answers: PCA (find the directions that matter) and clustering (find the natural groups). Both are exercises in seeing structure without being told what to look for.*

This expanded edition adds full step-by-step derivations wherever the original companion compressed the algebra, works every example through with real numbers you can check by hand or in a notebook, and adds “Worth Knowing” boxes that connect each idea to the tools and pitfalls you'll actually meet in practice. The two chapters share more machinery than they first appear to: both boil down to a constrained-optimization argument that surfaces eigenvectors as the answer — PCA finds them for a covariance matrix, spectral clustering finds them for a graph Laplacian, and the same Lagrange-multiplier trick derives both.

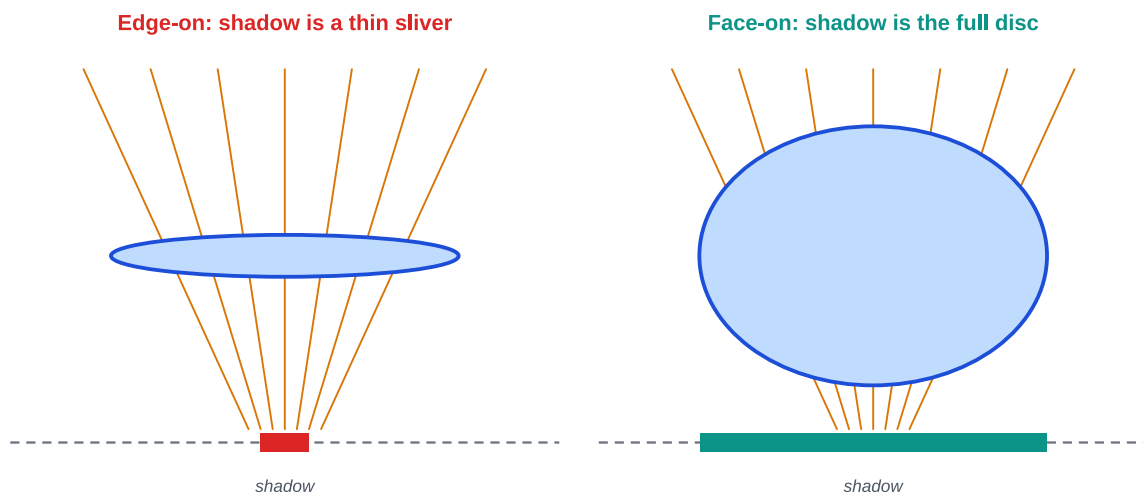
|                                                                                    |               |
|------------------------------------------------------------------------------------|---------------|
| <b>Chapter 12 — PCA, or Finding the Grain of the Wood .....</b>                    | <b>4</b>      |
| 12.1 The shadow analogy .....                                                      | 4             |
| 12.2 Finding the grain .....                                                       | 5             |
| 12.3 The mechanics, intuitively .....                                              | 5             |
| 12.4 A worked example, by hand .....                                               | 10            |
| 12.5 The crucial limitation — PCA is blind to your goal .....                      | 12            |
| 12.6 The nonlinear cousins .....                                                   | 14            |
| 12.7 Chapter 12 in one breath .....                                                | 16            |
| <br><b>Chapter 13 — K-Means &amp; Clustering, Finding the Natural Groups .....</b> | <br><b>17</b> |
| 13.1 The territory-drawing game .....                                              | 17            |
| 13.2 The objective it's minimizing .....                                           | 18            |
| 13.3 A worked example, step by step .....                                          | 20            |
| 13.4 The three ways k-means bites you (and the fixes) .....                        | 21            |
| 13.5 Clustering by shape — the picker's guide .....                                | 27            |
| 13.6 The connection to your work .....                                             | 27            |
| 13.7 Chapter 13 in one breath .....                                                | 30            |

## CHAPTER 12

# PCA, or Finding the Grain of the Wood

## 12.1 The shadow analogy

Imagine a 3D object — say, a flat dinner plate floating in space — and you want to describe it with as few numbers as possible. If you shine a light and cast its *shadow* onto a wall, the angle matters enormously. Shine it edge-on and the shadow is a thin, useless line. Shine it face-on and the shadow is a full circle that captures the plate's real shape. **PCA is the search for the best angle to cast the shadow** — the projection that preserves the most information about how the data is actually spread out.



**Figure 12.1:** Two projections of the same flat plate. The edge-on shadow collapses almost all the shape information into a sliver; the face-on shadow keeps it. PCA searches over every possible angle for the one that behaves like the face-on view.

“Information” here means **variance** — how spread out the data is. A direction along which the data varies a lot carries a lot of information; a direction where every point looks about the same carries almost none. PCA finds the directions of maximum variance and keeps those, discarding the low-variance directions as (probably) noise or redundancy.

## 12.2 Finding the grain

Another way to see it: wood has a grain — a natural direction along which it splits easily. Data often has a “grain” too: a primary direction along which most of the variation lies, then a secondary direction perpendicular to it, and so on. PCA finds these grain directions in order of importance, and the crucial subtlety is that *the grain is almost never aligned with your original coordinate axes*. If two features are correlated — taller people tend to weigh more, say — the cloud of data points tilts diagonally across the x–y plane, and neither the raw x-axis nor the raw y-axis on its own captures “how spread out” the data really is. PCA rotates the axes to match the cloud's actual shape: the first new axis (PC1) points along the tilt, the second (PC2) is perpendicular to it and picks up whatever variance is left over.

This is the point worth sitting with: PCA doesn't select from among your existing features. It *invents new axes* — each one a weighted combination of every original feature — chosen purely to fit the data's natural geometry, and then lets you keep only the top few.

### WORTH KNOWING — CORRELATION, COVARIANCE, AND “SPREAD” ARE THE SAME CONVERSATION

Covariance and correlation measure the same underlying relationship; correlation is just covariance rescaled to sit in  $[-1, 1]$  by dividing by the two features' standard deviations. PCA works directly on covariance (not correlation) unless you explicitly standardize your features first — a distinction that matters a lot in practice (see §12.3).

## 12.3 The mechanics, intuitively

The recipe, with the “why” attached:

1. **Center the data** (subtract the mean from every feature) — so we're measuring spread around the middle of the cloud, not around an arbitrary origin that has nothing to do with the data.
2. **Compute the covariance matrix** — a  $d \times d$  table capturing how every pair of features varies together; this is the mathematical description of the cloud's “shape.”
3. **Find its eigenvectors and eigenvalues** — the eigenvectors are the grain directions (the principal components); each eigenvalue tells you exactly how much variance lies along its direction.
4. **Sort by eigenvalue and keep the top  $k$**  — keep the directions that carry the most spread, drop the rest.

Step 3 is the one worth truly understanding rather than memorizing, because it isn't a heuristic — it's forced on us by the calculus of the problem we're actually solving.

**DERIVATION — VARIANCE MAXIMIZATION AND THE EIGENVECTOR CONDITION**

Let  $X$  be the centered data matrix ( $n$  rows,  $d$  columns — every column already has its mean subtracted). Projecting each row onto a unit direction  $w$  gives one number per point:  $z = Xw$ . Here  $\Sigma = X^T X/n$  is the covariance matrix.

**Step 1 — the variance of a 1-D projection**

$$\text{Var}(z) = (1/n) z^T z = (1/n)(Xw)^T (Xw) = w^T (X^T X/n) w = w^T \Sigma w$$

We want the direction that makes this variance as large as possible. Without the constraint  $\|w\|=1$ , the “maximum” is unbounded — you could just keep stretching  $w$ . Fixing its length turns this into a genuine eigenvalue problem instead of a degenerate one.

**Step 2 — pose it as a constrained optimization**

$$\text{maximize } w^T \Sigma w \quad \text{subject to } w^T w = 1$$

Introduce a Lagrange multiplier  $\lambda$  to fold the equality constraint into the objective.

**Step 3 — form the Lagrangian**

$$L(w, \lambda) = w^T \Sigma w - \lambda(w^T w - 1)$$

Using  $\nabla_w (w^T \Sigma w) = 2\Sigma w$  (valid because  $\Sigma$  is symmetric) and  $\nabla_w (w^T w) = 2w$ .

**Step 4 — differentiate and set to zero**

$$\nabla_w L = 2\Sigma w - 2\lambda w = 0 \Rightarrow \Sigma w = \lambda w$$

The stationary points of the maximization problem are, precisely, the eigenvectors of the covariance matrix. This is not a modeling choice or an approximation — it falls directly out of setting a derivative to zero.

**Step 5 — read off what this equation says**

*$w$  must be an eigenvector of  $\Sigma$ , and  $\lambda$  is its eigenvalue*

Substituting  $\Sigma w = \lambda w$  back into the original objective shows the variance captured along eigenvector  $w$  is exactly its eigenvalue  $\lambda$ . So to maximize variance, pick the eigenvector with the *largest* eigenvalue — that direction is PC1.

**Step 6 — recover the variance actually captured**

$$w^T \Sigma w = w^T (\lambda w) = \lambda(w^T w) = \lambda$$

For PC2, add an orthogonality constraint against  $w_1$ . Repeating exactly the same Lagrangian argument (now with two multipliers) again forces  $w$  to be an eigenvector of  $\Sigma$  — and among the remaining eigenvectors orthogonal to  $w_1$ , the one with the next-largest eigenvalue  $\lambda_2$  maximizes the leftover variance. Induction extends this argument to every subsequent component.

**Step 7 — extending to every later component**

*maximize  $w^T \Sigma w$  subject to  $w^T w = 1$  and  $w^T w_1 = 0$*

**The ordered eigenvectors of  $\Sigma$  —  $w_1, w_2, \dots, w_d$ , with eigenvalues  $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$  — are the principal components. The calculus and the geometry agree.**

In practice, we rarely form  $\Sigma$  and eigendecompose it directly. Instead, libraries compute the Singular Value Decomposition (SVD) of the centered data matrix itself, for reasons that are worth deriving rather than taking on faith.

**DERIVATION — FROM COVARIANCE EIGENDECOMPOSITION TO THE SVD**

$X$  is the  $n \times d$  centered data matrix.  $U$ 's columns are orthonormal “left” singular vectors,  $V$ 's columns are orthonormal “right” singular vectors, and  $S$  holds the singular values on its diagonal.

**Step 1 — write the SVD of the centered data**

$$X = USV^T, \quad U^T U = I, \quad V^T V = I, \quad S \text{ diagonal with } s_1 \geq s_2 \geq \dots \geq 0$$

Because  $U^T U = I$ , the  $U$  factors cancel completely, leaving an expression purely in terms of  $V$  and  $S$ .

**Step 2 — substitute into the covariance matrix**

$$\Sigma = X^T X / n = (USV^T)^T (USV^T) / n = VS^T U^T USV^T / n = VS^2 V^T / n$$

Compare against the standard form  $\Sigma = W \Lambda W^T$ , where  $W$  holds eigenvectors and  $\Lambda$  the eigenvalues on its diagonal. Matching term by term: the columns of  $V$  — the right singular vectors of  $X$  — ARE the eigenvectors of  $\Sigma$  (the principal components), and the eigenvalues are  $\lambda_i = s_i^2 / n$ .

**Step 3 — match it to an eigendecomposition**

$$\Sigma = V(S^2/n)V^T \text{ is exactly an eigendecomposition of } \Sigma$$

**PCA via SVD: run the SVD directly on the centered data matrix  $X$ . Its right singular vectors  $V$  are the principal components, and  $\lambda_i = s_i^2/n$  gives the variance each one captures — with no need to ever explicitly form  $\Sigma = X^T X$ .**

The SVD route isn't just more elegant — it's more numerically trustworthy. Forming  $X^T X$  squares every singular value, which squares the matrix's *condition number* (the ratio of its largest to smallest singular value). For data with highly correlated features, that squaring can amplify floating-point rounding error enough to blur or flip the smaller eigenvectors. Running SVD on  $X$  directly sidesteps the squaring entirely, which is why libraries such as scikit-learn's `PCA` compute an SVD under the hood rather than eigendecomposing an explicit covariance matrix.



**INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES**

***“Why does PCA use the eigenvectors of the covariance matrix, and what does the eigenvalue represent?”***

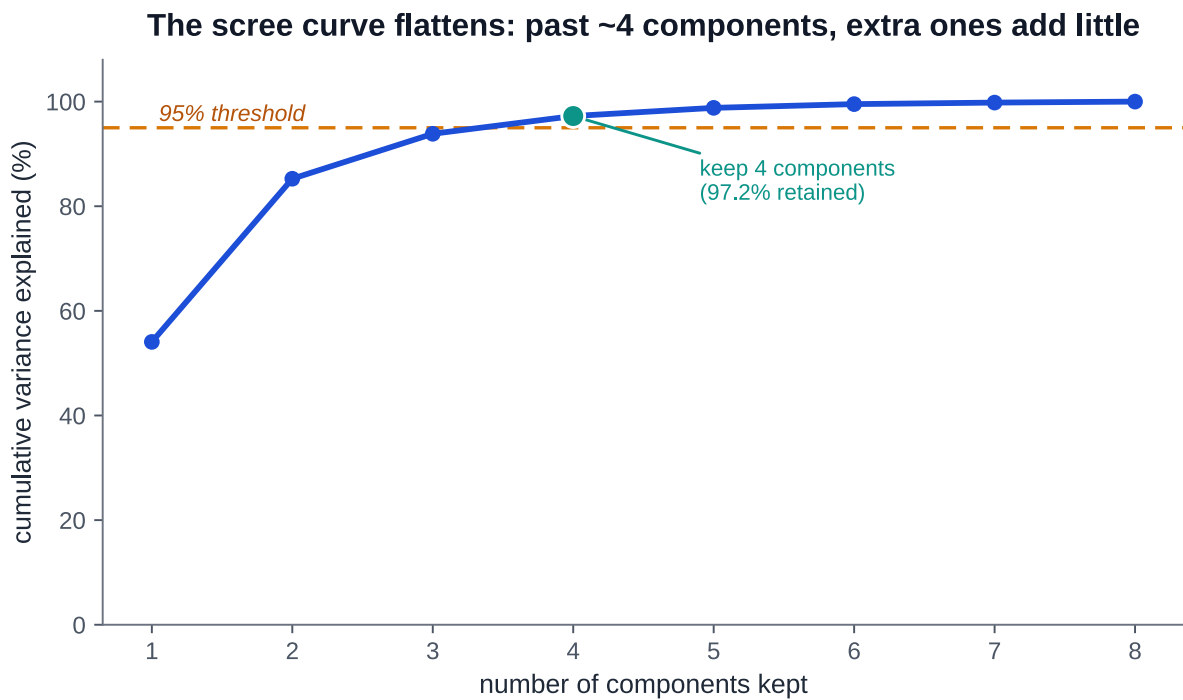
This is exactly §12.3: maximizing the variance of a 1-D projection  $w^T \Sigma w$  subject to  $\|w\|=1$  is a Lagrange-multiplier problem whose stationary condition is  $\Sigma w = \lambda w$  — the definition of an eigenvector/eigenvalue pair. Plugging that back into the objective shows the eigenvalue  $\lambda$  literally equals the variance captured along that direction, which is why sorting by eigenvalue and keeping the top  $k$  is the right way to choose which directions to retain.

**WORTH KNOWING — STANDARDIZE BEFORE YOU DECORRELATE**

PCA maximizes variance in whatever units your features happen to be in. If one feature is measured in millimeters and another in kilometers, the millimeter feature's numerically tiny variance will always lose to the kilometer feature's numerically huge variance — regardless of which one is more informative. In practice, run something like scikit-learn's

`StandardScaler` (zero mean, unit variance per feature) before PCA unless every feature is already on a comparable, meaningful scale. scikit-learn's `PCA` class does *not* do this for you automatically — it only centers the data, it doesn't rescale it.

**Choosing  $k$ :** plot the cumulative variance captured as you add components, and stop once you've retained “enough” (often 90–95%). This is the scree plot / explained-variance approach, and it's worth seeing on real (if synthetic) numbers rather than a schematic curve.



**Figure 12.2:** An actual PCA run on an 8-feature synthetic dataset built with a deliberately decreasing variance schedule. The curve climbs fast at first, then flattens — the classic signature that lets you read off “enough components” directly from the plot: here, 4 of the 8 components already capture 97.2% of the total variance.

## 12.4 A worked example, by hand

Take four points in two dimensions:

| Point | x | y |
|-------|---|---|
| A     | 2 | 1 |
| B     | 3 | 4 |
| C     | 5 | 5 |
| D     | 6 | 2 |

**Step 1 — center the data.** The mean is  $\bar{x} = (2+3+5+6)/4 = 4$  and  $\bar{y} = (1+4+5+2)/4 = 3$ , so the mean point is (4, 3). Subtracting it from every point gives the centered coordinates A'(-2,-2), B'(-1,1), C'(1,2), D'(2,-1).

**Step 2 — compute the (sample) covariance matrix,** using  $n-1 = 3$  in the denominator:

$$\sigma_{xx} = ((-2)^2 + (-1)^2 + 1^2 + 2^2)/3 = 10/3 \approx 3.33 \quad \sigma_{yy} = ((-2)^2 + 1^2 + 2^2 + (-1)^2)/3 = 10/3 \approx 3.33$$

$$\sigma_{xy} = ((-2)(-2) + (-1)(1) + (1)(2) + (2)(-1))/3 = 3/3 = 1$$

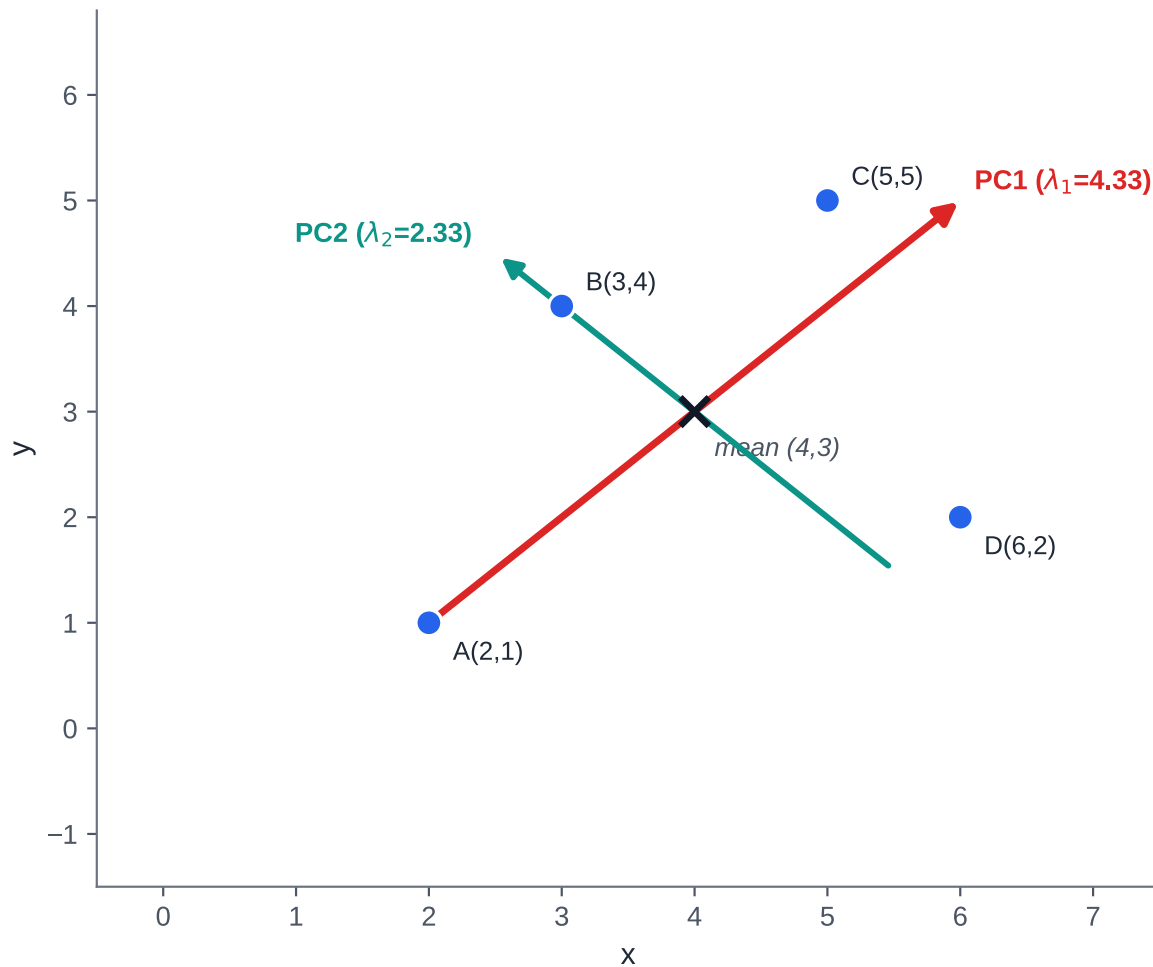
giving  $\Sigma = [[3.33, 1], [1, 3.33]]$ . **Step 3 — find the eigenvalues.** Because the two diagonal entries are equal (call the shared value  $a = 3.33$  and the off-diagonal  $b = 1$ ), the characteristic equation  $(a-\lambda)^2 - b^2 = 0$  factors cleanly into  $\lambda = a \pm b$ , giving  $\lambda_1 = 4.33$  and  $\lambda_2 = 2.33$  — and for this equal-diagonal shape, the corresponding eigenvectors are always the  $45^\circ$  directions  $(1,1)/\sqrt{2}$  and  $(1,-1)/\sqrt{2}$ .

PC1 = (0.707, 0.707),  $\lambda_1 = 4.33$  (65% of the variance) — PC2 = (0.707, -0.707),  $\lambda_2 = 2.33$  (35% of the variance)

**Step 4 — project onto PC1** by taking the dot product of each centered point with (0.707, 0.707):

| Point | centered (x, y) | projection onto PC1 |
|-------|-----------------|---------------------|
| A     | (-2, -2)        | -2.83               |
| B     | (-1, 1)         | 0.00                |
| C     | (1, 2)          | 2.12                |
| D     | (2, -1)         | 0.71                |

### The worked example: PC1 aligns with the blob's diagonal tilt

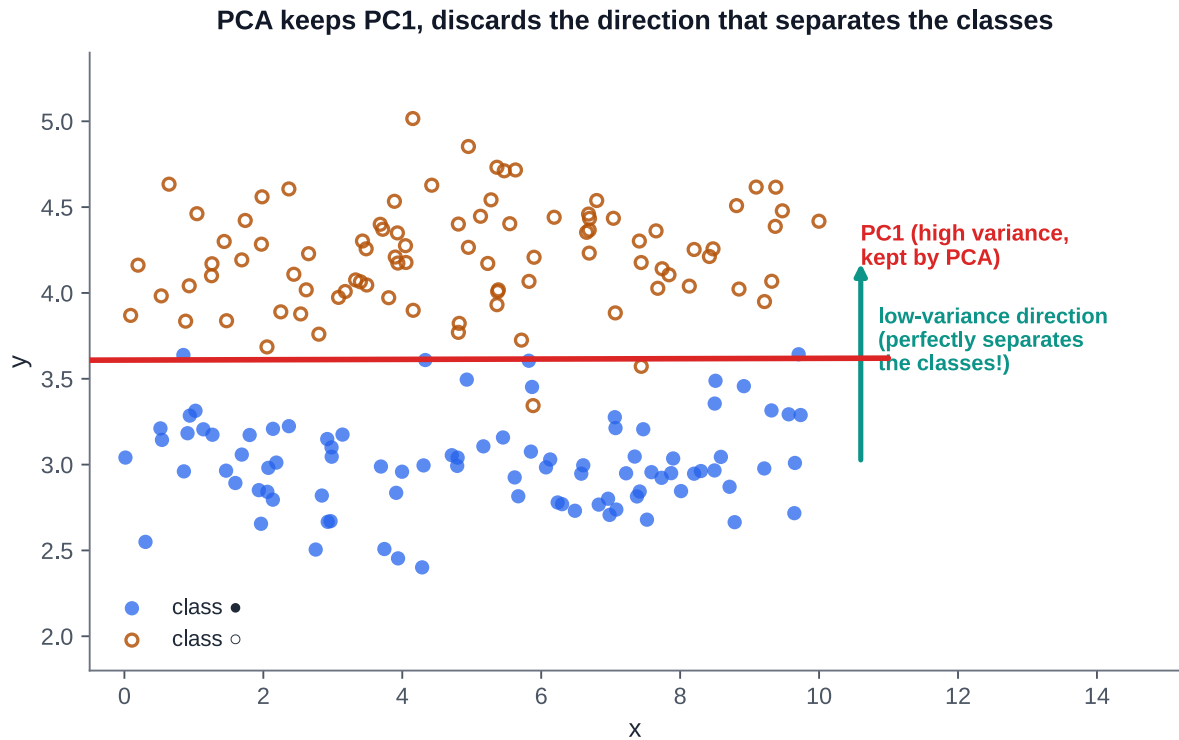


**Figure 12.3:** The exact worked example above, plotted. PC1 doesn't align with the raw x- or y-axis — it aligns with the diagonal tilt of the cloud, exactly as §12.2 predicted. Arrow lengths are scaled by each component's standard deviation ( $\sqrt{\lambda}$ ) to show how much of the spread each one actually carries.

Notice the numbers agree with the geometry: A and D sit toward the lower-left of the tilt and get negative or small projections, while C sits toward the upper-right and gets the largest positive projection. Collapsing these four 2-D points down to these four 1-D numbers keeps 65% of the original variance — a real compression, with a precisely quantified cost.

## 12.5 The crucial limitation — PCA is blind to your goal

Here's the subtle, interview-worthy catch: **PCA maximizes variance, which is not always the same as keeping what's useful.** PCA is *unsupervised* — it never looks at your labels. So it can happily throw away a low-variance direction that happens to be exactly the one that predicts your target.



**Figure 12.4:** A genuinely computed example: both classes share the same wide spread along the  $x$ -axis (high variance, but useless for telling them apart), while a narrow band along  $y$  perfectly separates them. Running real PCA on this data confirms PC1 aligns almost exactly with  $x$  ( $\lambda_1/\lambda_2 \approx 17:1$ ) — PCA would keep the useless direction and discard the perfectly discriminative one.

So PCA is a *compression* and *denoising* tool, not a *prediction* tool. Use it to speed up downstream models, decorrelate features, or visualize structure — but don't assume the top components are automatically the most *predictive* ones. When what you actually want is the direction that best *separates classes* (rather than the direction of maximum spread), that's a different, supervised technique: Linear Discriminant Analysis (LDA). Where PCA maximizes total variance  $w^T \Sigma w$ , LDA maximizes the ratio of between-class variance to within-class variance,

$$J(w) = (w^T S_B w) / (w^T S_W w)$$

where  $S_B$  measures how far apart the class means are and  $S_W$  measures how spread out each class is internally — a direction can win on this ratio precisely because the classes are tightly packed and far apart along it, even if its raw variance is small in the PCA sense.

**WORTH KNOWING — KERNEL PCA — THE SAME TRICK, ONE MORE TIME**

If PCA's blind spot is that it only finds *linear* directions, one fix that stays fully unsupervised is kernel PCA: implicitly map the data into a much higher-dimensional space via a kernel function (the same trick used by kernelized SVMs), then run ordinary linear PCA in that space. Directions that are curved in the original feature space can become linear in the lifted space, letting kernel PCA capture nonlinear structure without ever explicitly computing the high-dimensional coordinates.

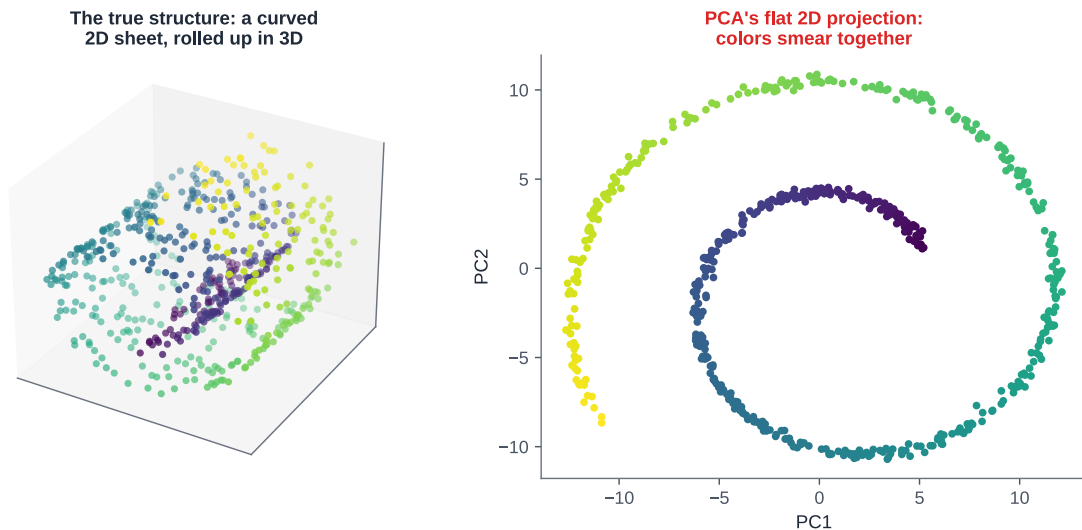
**INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES**

***“If you're about to train a classifier, is it ever a bad idea to run PCA first?”***

Yes — this is exactly §12.5. PCA keeps the directions of highest variance regardless of whether they help separate your classes, so on data shaped like the figure above, PCA-then-classify can throw away the one direction that made the problem easy. If class separability is the actual goal, LDA (or simply not reducing dimensionality at all) is often a better first move than PCA.

## 12.6 The nonlinear cousins

PCA can only find *straight* grain — linear directions through the data. If your data lies on a curved surface (a “manifold,” like a rolled-up sheet), PCA flattens it clumsily, because any straight-line projection is fundamentally the wrong shape of tool for a curved object.



**Figure 12.5:** A real “Swiss roll” dataset (points on a curved 2-D sheet, embedded and rolled up in 3-D) and its actual top-2 PCA projection. Points that are far apart along the roll (different colors) end up next to each other in the flattened projection — PCA can only slice through the roll with a flat plane, it cannot unroll it.

For data like this, the usual alternatives are:

- **t-SNE** — brilliant for *visualizing* clusters in 2D by converting pairwise similarities into probabilities and matching them in the low-dimensional map (minimizing a KL divergence between the two similarity distributions). It's excellent at revealing local cluster structure, but the resulting distances *between* clusters aren't meaningful, results are sensitive to the perplexity hyperparameter, and it isn't a reusable transform — you can't drop a new point into an existing t-SNE map without recomputing everything. A picture, not a tool.
- **UMAP** — built on ideas from topological data analysis (approximating the data as a “fuzzy” simplicial complex), UMAP is typically faster than t-SNE, tends to preserve more global structure (relative distances between clusters carry more meaning), and — unlike t-SNE — can transform new, unseen data through the same learned embedding.
- **Autoencoders** — neural networks that learn a nonlinear compression by forcing information through a narrow “bottleneck” layer and training to reconstruct the input from it. In fact, PCA is exactly what you get from a linear autoencoder with a single hidden layer and squared-error loss: with no nonlinearity anywhere, the weights that minimize reconstruction error span the same subspace as the top-k principal components. Add nonlinear activations, and the network can learn to unroll a manifold the way PCA never could.

**WORTH KNOWING — WHERE THESE ACTUALLY GET USED**

t-SNE and UMAP are the default tools for visualizing high-dimensional embeddings — word/sentence embeddings, single-cell genomics data, learned representations from a trained network — almost always as a final 2-D or 3-D picture for humans to look at, not as a preprocessing step feeding into another model. If you need a reusable, model-ready nonlinear reduction, UMAP or an autoencoder are the more common production choices; t-SNE is reached for almost exclusively for the plot itself.

**INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES**

*“Why can't you use t-SNE the same way you'd use PCA, as a general preprocessing step before another model?”*

This is exactly §12.6: t-SNE optimizes a layout for a fixed dataset by matching similarity distributions, it has no notion of a reusable mapping you can apply to a new point, and the distances it produces between clusters (as opposed to within them) are not meaningful. PCA, by contrast, is a genuine linear transform  $w$  — you can apply the same  $w$  to new data forever, which is exactly what a preprocessing step needs.

## 12.7 Chapter 12 in one breath

**CHAPTER 12 IN ONE BREATH**

PCA finds the best “angle” to project data — the directions of maximum variance (the data's natural grain) — by solving a constrained variance-maximization problem whose solution is forced, by Lagrange multipliers, to be the eigenvectors of the covariance matrix, with each eigenvalue equal to the variance that direction captures; in practice these come from an SVD of the centered data for numerical stability. Its blind spot: being unsupervised, it maximizes variance, not predictive usefulness, so it can discard a low-variance-but-highly-predictive direction (LDA is the supervised fix). It's fundamentally linear; t-SNE, UMAP, kernel PCA, and autoencoders handle the curved structure it cannot.



## CHAPTER 13

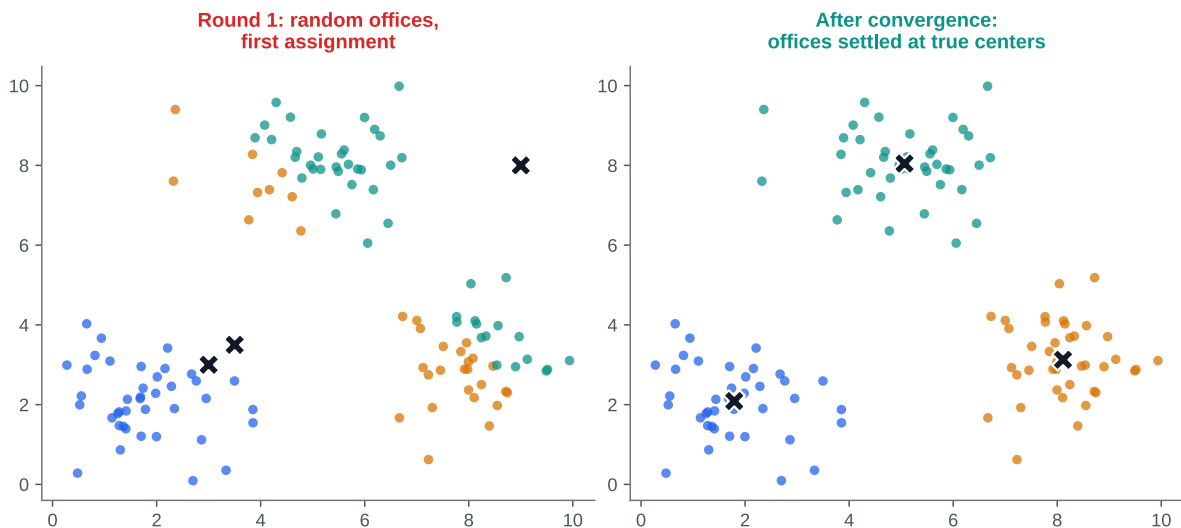
# K-Means & Clustering, Finding the Natural Groups

## 13.1 The territory-drawing game

You're dropped into a country with cities scattered across it, and asked to divide it into  $k$  regions, each served by one post office, so that every city is close to its region's office. Where do you put the offices, and which cities belong to which?

K-means solves this with a simple back-and-forth (the algorithm is formally known as Lloyd's algorithm):

1. Drop  $k$  post offices randomly.
2. **Assign** each city to its nearest office.
3. **Move** each office to the center (mean) of the cities currently assigned to it.
4. Repeat steps 2–3 until nothing changes.



**Figure 13.1:** A real  $k$ -means run on synthetic data. Left: the first assignment from randomly placed offices (marked  $\times$ ). Right: the same algorithm after running to convergence — the offices have drifted to the true centers of their clusters and every city's assignment has stabilized.

Each round, cities get reassigned and offices drift toward their cluster's true center. It's guaranteed to settle — each step reduces (or leaves unchanged) the total city-to-office distance — this “assign, then re-center, repeat” pattern is *coordinate descent* on the clustering objective, and §13.2 derives exactly why that guarantees convergence.

## 13.2 The objective it's minimizing

Formally, k-means minimizes the total squared distance from each point to its cluster's center (the “inertia”):

$$J(C, \{\mu_k\}) = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Every step of the algorithm lowers this number, which is why it always converges. But — critically — it converges to a *local* minimum, not necessarily the best possible one. The derivation below makes precise exactly what each of the two alternating steps is optimizing, and why alternating between them is guaranteed to terminate.

**DERIVATION — K-MEANS AS ALTERNATING MINIMIZATION**

$C = \{C_1, \dots, C_K\}$  is the partition of points into clusters, and  $\mu_k$  is cluster  $k$ 's center. This is the total squared distance from every point to its own cluster's center.

**Step 1 — the objective, restated**

$$J(C, \{\mu_k\}) = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

With every  $\mu_k$  held fixed,  $J$  splits into one independent term per point (each point appears in exactly one inner sum), so minimizing the whole sum reduces to minimizing each point's own term separately — trivially achieved by assigning  $x_i$  to its nearest center.

**Step 2 — fix the centers, optimize the assignment**

for each  $x_i$ , choose the  $k$  that minimizes  $\|x_i - \mu_k\|^2$

With the assignment  $C$  now fixed, each  $\mu_k$  appears only in its own inner sum. Differentiating that sum with respect to  $\mu_k$  and setting it to zero shows the minimizing center is exactly the mean of the points currently assigned to it — hence “re-center to the average.”

**Step 3 — fix the assignment, optimize the centers**

$$\partial J / \partial \mu_k = -2 \sum_{x_i \in C_k} (x_i - \mu_k) = 0 \Rightarrow \mu_k = (1/|C_k|) \sum_{x_i \in C_k} x_i$$

Step 2 can only lower  $J$  (each point moves to an equally-good-or-better center than the one it had). Step 3 can only lower  $J$  (the mean is the unique minimizer of a sum of squared distances, proven in Step 3). Chaining the two shows the sequence of  $J$  values is non-increasing.

**Step 4 — why  $J$  never increases**

$$J(C^{(t+1)}, \mu^{(t)}) \leq J(C^{(t)}, \mu^{(t)}) \quad \text{and} \quad J(C^{(t+1)}, \mu^{(t+1)}) \leq J(C^{(t+1)}, \mu^{(t)})$$

A non-increasing sequence of real numbers, drawn from a finite set of possible values (one value per possible partition of the data), cannot decrease forever. It must eventually stop changing — at which point no point gets reassigned and the algorithm has converged.

**Step 5 — why it actually stops**

$J$  is bounded below by 0, and there are only finitely many ways to partition  $n$  points into  $k$  groups

**K-means is coordinate descent on  $J(C, \mu)$ : alternately minimizing over the assignment, then over the centers. It is guaranteed to converge to A local minimum of  $J$  — but because  $J$  is genuinely non-convex in the**

**(combinatorial, not continuous) assignment variable, different starting centers can converge to different local minima, some far worse than others.**

**INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES**

**“Does *k*-means always converge? Does it always find the best clustering?”**

This is exactly §13.2: it always converges, because each of its two alternating steps can only decrease (never increase) the inertia  $J$ , and  $J$  is bounded below with only finitely many achievable values. It does *not* always find the global optimum, because the assignment step is a discrete, non-convex choice — alternating minimization on a non-convex objective is only guaranteed to reach a local minimum, and which one depends on where you start (§13.4, Bite 1).

## 13.3 A worked example, step by step

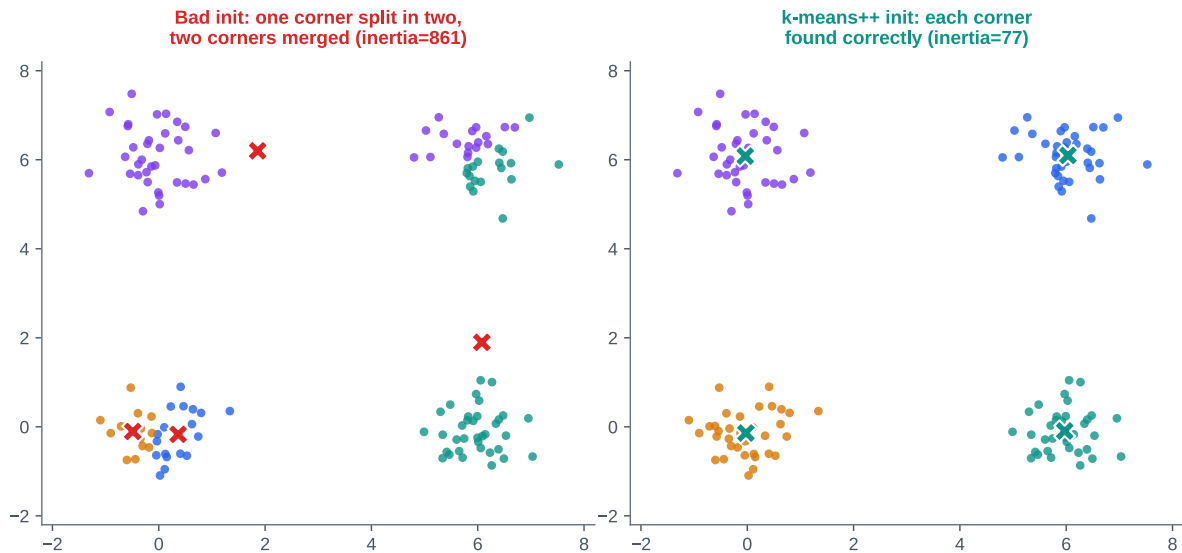
Six points on a number line:  $\{1, 2, 3, 9, 10, 11\}$ , with  $k = 2$  and a deliberately imperfect starting guess for the two centers:  $\mu_1 = 2$ ,  $\mu_2 = 3$  (both dropped inside the left group).

| Round | $\mu_1$ | $\mu_2$ | assignment (dist. to $\mu_1$ vs $\mu_2$ )                                                                            | new $\mu_1, \mu_2$       | inertia |
|-------|---------|---------|----------------------------------------------------------------------------------------------------------------------|--------------------------|---------|
| 1     | 2       | 3       | $1 \rightarrow C_1, 2 \rightarrow C_1, 3 \rightarrow C_2, 9 \rightarrow C_2, 10 \rightarrow C_2, 11 \rightarrow C_2$ | 1.5, 8.25                | 150.0   |
| 2     | 1.5     | 8.25    | $1 \rightarrow C_1, 2 \rightarrow C_1, 3 \rightarrow C_1, 9 \rightarrow C_2, 10 \rightarrow C_2, 11 \rightarrow C_2$ | 2.0, 10.0                | 13.94   |
| 3     | 2.0     | 10.0    | (same as round 2 — stable)                                                                                           | 2.0, 10.0<br>(converged) | 4.0     |

Two things are worth pausing on. First, the imperfect start ( $\mu_2 = 3$  sitting right in the middle of the left group) doesn't doom the run — by round 2 the algorithm has already found the correct split  $\{1,2,3\} \mid \{9,10,11\}$ , illustrating that *k*-means can recover from a merely *suboptimal* start even though (as §13.4 shows) it can't always recover from a genuinely *bad* one. Second, watch the inertia column:  $150.0 \rightarrow 13.94 \rightarrow 4.0$ , strictly decreasing at every step exactly as §13.2's derivation guarantees — and 4.0 is in fact the lowest inertia achievable for this data at  $k=2$ , since the two natural groups are  $\{1,2,3\}$  (variance-around-mean of 2.0) and  $\{9,10,11\}$  (variance-around-mean of 2.0), summing to 4.0.

## 13.4 The three ways k-means bites you (and the fixes)

**Bite 1 — bad luck in the starting positions.** Where you drop the initial offices matters enormously; a bad start can land you in a poor local minimum where, say, one true cluster gets split in two while two other true clusters get merged into one.



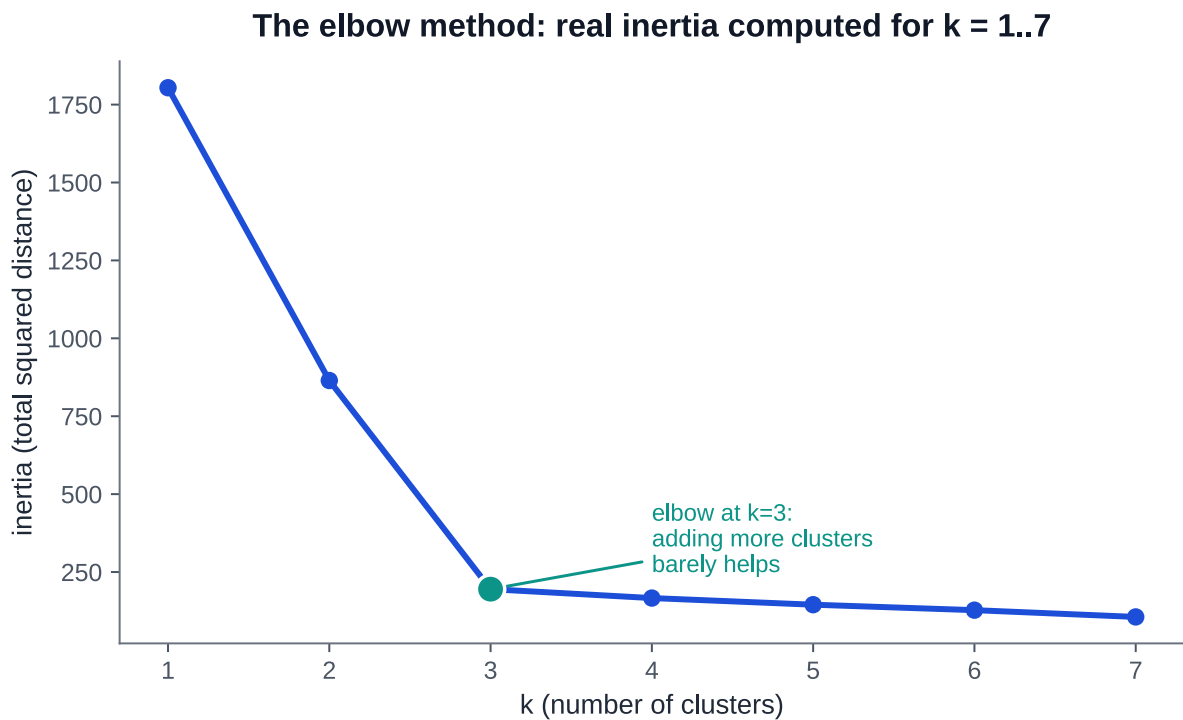
**Figure 13.2:** Same data, same  $k$ , same algorithm — two different starting points. The bad init (left) genuinely converges (inertia 861) with one corner cluster wastefully split in two and two others merged; the spread-out init (right) converges to the correct answer (inertia 77). Both are real, computed fixed points of the same algorithm.

**Fix — k-means++:** don't drop offices purely at random; place each new one *far* from the existing ones. Concretely, after choosing the first center uniformly at random, each subsequent center is sampled with probability proportional to its squared distance from the nearest center already chosen:

$$P(x \text{ chosen as next center}) = D(x)^2 / \sum_x D(x')^2 \quad \text{where } D(x) = \text{distance from } x \text{ to the nearest already-chosen center}$$

This spreads the initial offices out sensibly (points far from every existing center are far more likely to be picked next) and is now the standard default — scikit-learn's `KMeans` uses `init='k-means++'` by default. It's also standard practice to run the whole algorithm several times with different random starts (scikit-learn's `n_init` parameter) and keep whichever run achieves the lowest final inertia.

**Bite 2 — you have to guess  $k$  in advance.** The algorithm has no way to tell you how many groups actually exist.



**Figure 13.3:** Real inertia computed by actually running *k*-means for *k* = 1 through 7 on data generated from 3 true clusters. The curve drops steeply up to *k*=3, then flattens — the “elbow” — telling you additional clusters are mostly just subdividing real structure rather than finding new structure.

**Fix — the elbow method**, shown above, or the **silhouette score**, which measures how snugly each point fits its own cluster versus the next-nearest one:

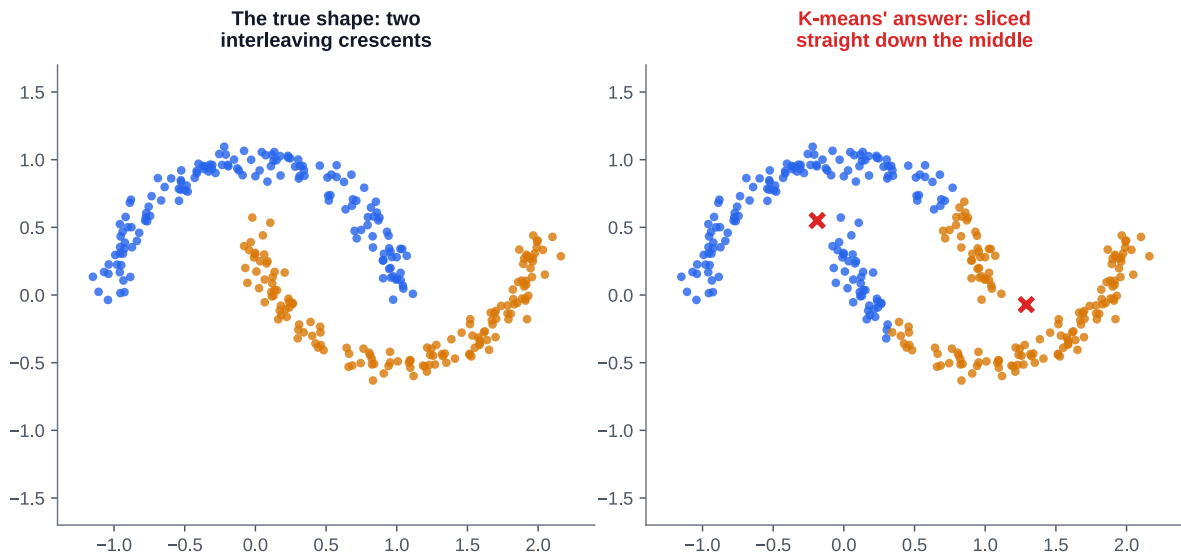
$$s(i) = (b(i) - a(i)) / \max(a(i), b(i))$$

where *a*(*i*) is the mean distance from point *i* to every other point in its *own* cluster, and *b*(*i*) is the mean distance from *i* to every point in the *nearest other* cluster.

#### WORTH KNOWING — READING A SILHOUETTE SCORE

*s*(*i*) ranges from −1 to 1. Values near +1 mean the point sits much closer to its own cluster than to any other (*a*(*i*) ≪ *b*(*i*)) — a confident assignment. Values near 0 mean the point is roughly equidistant from two clusters, sitting right on a boundary. Negative values mean the point is actually closer, on average, to a different cluster than the one it was assigned to — a warning sign that *k* is wrong or that cluster is a poor fit. Averaging *s*(*i*) over all points gives a single per-*k* score you can maximize (e.g. via scikit-learn's `silhouette_score`) as an alternative to eyeballing an elbow plot.

**Bite 3 — it assumes round, equal-sized blobs.** K-means assigns points using distance-to-center, which means every cluster it can represent is, geometrically, a Voronoi cell around a single point — roughly spherical and comparably sized. Give it two long crescents, or a small dense cluster next to a big sparse one, and it carves them wrongly, no matter how many times you restart it.



**Figure 13.4:** A real  $k$ -means run ( $k=2$ ) on two interleaving crescent-shaped clusters. Because  $k$ -means can only draw a straight decision boundary between two centers, it slices the two crescents down the middle instead of following their true curved shape — no amount of re-initialization fixes this, because the problem is the shape of boundary  $k$ -means is capable of drawing, not where it starts.

**Fix — switch tools.** A **Gaussian Mixture Model (GMM)** allows stretched, elliptical clusters and soft “60% this cluster, 40% that” memberships, by modeling the data as a weighted sum of Gaussian distributions and fitting via the Expectation-Maximization (EM) algorithm rather than hard assignment. **DBSCAN** finds arbitrary shapes by growing clusters through dense regions and labels sparse outliers as noise — and doesn't need  $k$  up front at all.

#### WORTH KNOWING — GMM'S SOFT ASSIGNMENTS, IN ONE LINE

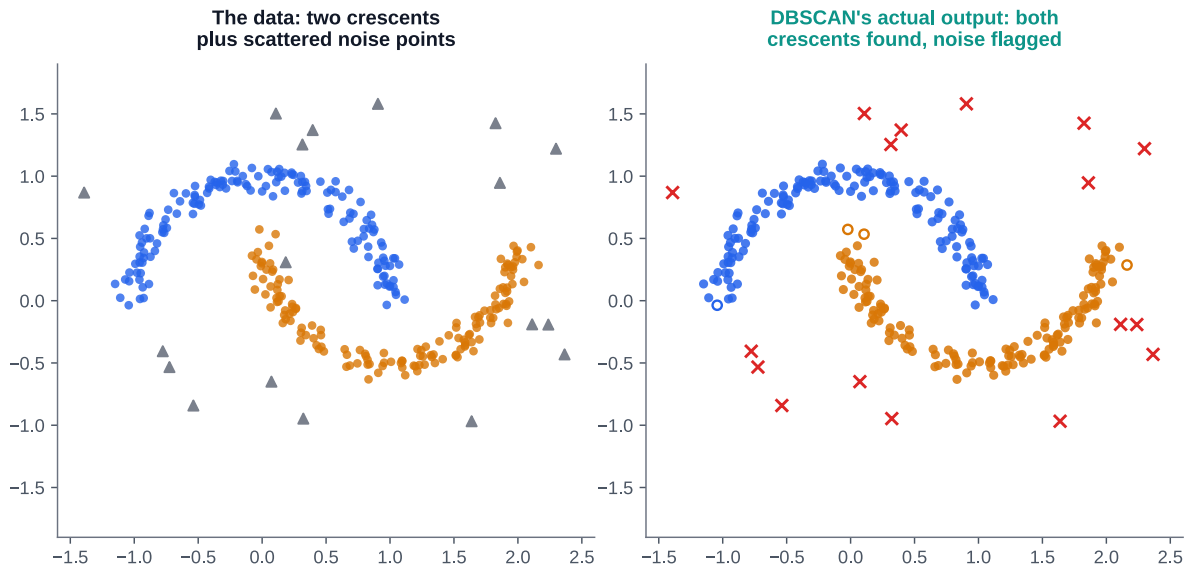
A GMM assigns each point a *responsibility*  $\gamma_{nk}$  — the posterior probability that point  $n$  came from Gaussian component  $k$ , computed via Bayes' rule from each component's mean, covariance, and mixing weight. The E-step computes these responsibilities given the current parameters; the M-step re-estimates each component's mean, covariance, and weight as a responsibility-weighted average over the data — a soft, probabilistic echo of  $k$ -means' assign/re-center loop, with elliptical (not just spherical) clusters as a direct consequence of each component having its own full covariance matrix.

**DBSCAN, in full.** DBSCAN (Density-Based Spatial Clustering of Applications with Noise) needs two parameters:  $\epsilon$  (a radius) and `minPts` (a density threshold). Everything else follows from three formal definitions built on top of them:

- The  **$\epsilon$ -neighborhood** of a point  $p$  is  $N_\epsilon(p) = \{q \in D : \text{dist}(p, q) \leq \epsilon\}$  — every point within radius  $\epsilon$  of  $p$ .
- $p$  is a **core point** if  $|N_\epsilon(p)| \geq \text{minPts}$  — it has enough neighbors nearby to sit inside a dense region. A point  $q$  is *directly density-reachable* from a core point  $p$  if  $q \in N_\epsilon(p)$ .
- $q$  is **density-reachable** from  $p$  if there's a chain  $p = p_1, p_2, \dots, p_n = q$  where each  $p_{i+1}$  is directly density-reachable from  $p_i$ . This relation is *not* symmetric: a border point can be density-reachable from a core point without itself being a core point, so the chain can't necessarily run backward through it.
- $p$  and  $q$  are **density-connected** if some point  $o$  exists with both  $p$  and  $q$  density-reachable from  $o$ . Unlike density-reachability, this relation *is* symmetric — and a cluster is defined formally as a maximal set of mutually density-connected points. Anything left over — not density-reachable from any core point at all — is **noise**.

The algorithm itself is a direct implementation of those definitions: visit each point; if it's a core point, start a new cluster and grow it by repeatedly pulling in every point density-reachable from it (whenever that newly-added point is itself a core point, its neighbors get pulled in too — a breadth-first expansion through the dense region); points that never get pulled into any cluster this way are left labeled noise. Because clusters are built by chaining together overlapping  $\epsilon$ -balls rather than measuring distance to one fixed center, the resulting shape follows wherever the density actually leads — a crescent, a spiral, a ring — instead of the single straight cut a center-based method like k-means is limited to.



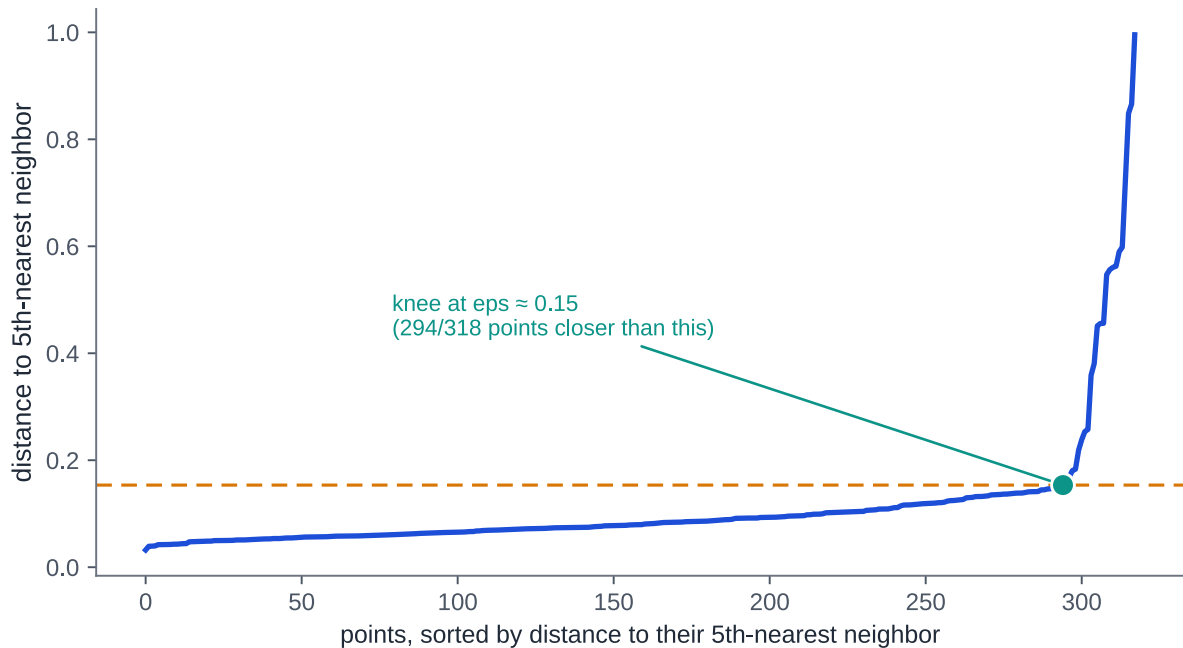


**Figure 13.6:** A real, fitted DBSCAN run ( $\epsilon=0.18$ ,  $\text{minPts}=5$ ) on the same two crescents, now with 18 scattered noise points added. DBSCAN recovers both crescents exactly (150/150 points each, shown solid for core points and hollow for border points) and correctly flags 17 of the 18 true noise points (red  $\times$ ) as noise — something no amount of re-initializing  $k$ -means could ever do, since  $k$ -means has no concept of a point simply not belonging to any cluster.

#### WORTH KNOWING — CHOOSING EPS AND MINPTS

A common rule of thumb for `minPts` is to set it to at least  $D + 1$ , where  $D$  is the number of features (so a 2-D problem rarely uses fewer than `minPts=4` or 5); noisier data or higher dimensions call for a larger value. Once `minPts` is fixed, `eps` is usually chosen from a **k-distance graph**: compute every point's distance to its `minPts`-th nearest neighbor, sort those distances, and plot them. Points in dense regions have a small, slowly-changing  $k$ -th-neighbor distance; once you run out of dense region and start hitting sparse or isolated points, that distance jumps sharply — and the “knee” where the curve bends upward is a principled choice for `eps`, playing exactly the same visual role here that the elbow does for choosing  $k$  in §13.4, Bite 2.

### Choosing eps: the k-distance graph's knee



**Figure 13.7:** The actual  $k$ -distance graph ( $\text{minPts}=5$ ) for the crescents-plus-noise data above. The curve stays low and nearly flat while it's still walking through the two dense crescents, then rockets upward once it reaches the sparse, scattered points — the knee at  $\text{eps} \approx 0.15$  sits almost exactly at the true boundary between signal and noise, and is very close to the  $\text{eps}=0.18$  actually used to fit Figure 13.6.

#### WORTH KNOWING — DBSCAN'S OWN BLIND SPOT

DBSCAN isn't a universal fix. Because it applies one global  $\epsilon$  and  $\text{minPts}$  everywhere, a dataset with clusters of genuinely different densities can defeat it: a setting loose enough to keep a sparse (but real) cluster from being called noise is often loose enough to silently merge two denser clusters that happen to sit close together. HDBSCAN (hierarchical DBSCAN) addresses this by examining a whole range of density thresholds at once rather than committing to a single  $\epsilon$ . DBSCAN also inherits the usual curse-of-dimensionality problem that affects any distance-based method: in high dimensions, distances between points tend to concentrate (the ratio of nearest to farthest distance creeps toward 1), which erodes exactly the density contrast DBSCAN relies on to tell a core point from noise. In practice this often means running PCA (§12.3) or another dimensionality reduction step first, before DBSCAN ever sees the data.

**INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES**

***“In DBSCAN, is “density-reachable” a symmetric relationship? Why does it matter?”***

No, and this is exactly the formal definitions earlier in §13.4: a border point can be density-reachable FROM a core point (it's within  $\epsilon$  of it) without being able to reach back, because the border point itself may not have enough neighbors to qualify as core. That's precisely why the cluster definition is built on density-*connectivity* (via some shared core point o) rather than density-reachability directly — connectivity is symmetric, which is what lets “cluster” be a well-defined, order-independent partition of the data instead of depending on which point you happened to start growing from.

**INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES**

***“You're given a dataset with two interleaving crescent-shaped clusters. Would you use k-means?”***

No — this is exactly §13.4, Bite 3. K-means can only carve up space with straight-line (Voronoi) boundaries around cluster centers, so it will slice through the crescents rather than follow their curve, regardless of  $k$  or initialization. DBSCAN (density-based, no assumption of a single center per cluster, as Figure 13.6 shows directly) or spectral clustering (§13.6) are the standard fixes for exactly this shape of problem.

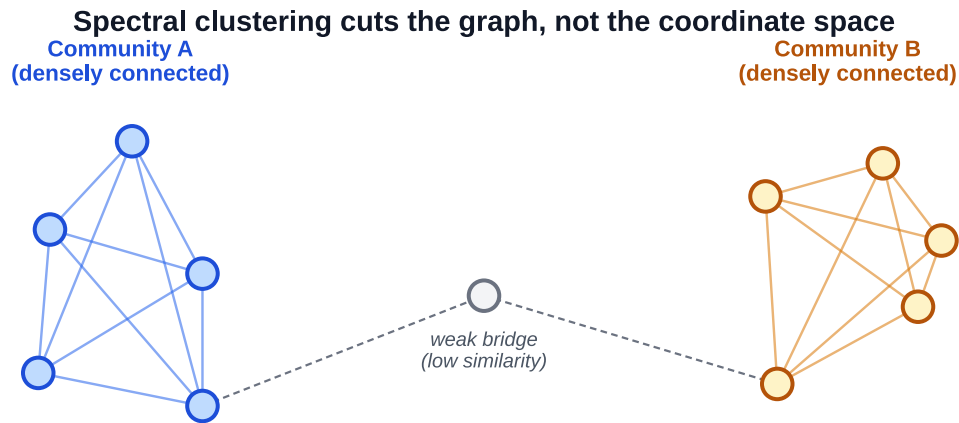
## 13.5 Clustering by shape — the picker's guide

| If clusters are...               | use...                      |
|----------------------------------|-----------------------------|
| round, similar size              | K-means (fast, simple)      |
| elliptical, want soft membership | GMM (probabilistic)         |
| arbitrary shape + outliers       | DBSCAN (density-based)      |
| unknown $k$ , want a hierarchy   | Hierarchical (dendrogram)   |
| defined by connectivity          | Spectral (clusters a graph) |

## 13.6 The connection to your work

That last row — **spectral clustering** — is your bridge. It doesn't cluster in raw feature space with Euclidean distance; it builds a *similarity graph* (who's connected to whom) and clusters *that*. This is exactly the mindset behind graph/knowledge-graph work where entities are better grouped by their web

of relationships (connectivity) than by raw distance in a feature vector: being able to say “*I chose graph-based clustering because the relevant similarity was relational, not geometric — Euclidean distance in feature space would have missed the structure*” demonstrates you pick clustering methods by matching the algorithm's assumptions to the data's true shape. That judgment is what separates applying a tool from understanding it.



**Figure 13.5:** Two densely-connected communities joined by only a thin, weak bridge. Spectral clustering is built to find exactly this kind of structure — it looks for the cheapest way to cut the graph into pieces, not the tightest way to circle points in space.

The mechanics behind that cut are themselves a beautiful echo of §12.3's Lagrange-multiplier argument — only this time the matrix being eigendecomposed describes a graph instead of a data cloud.

### DERIVATION — THE GRAPH LAPLACIAN, THE RAYLEIGH QUOTIENT, AND THE FIEDLER VECTOR

$W$  is the symmetric similarity/adjacency matrix of the graph — a weighted edge between two entities that are strongly related.  $D$  is the diagonal “degree” matrix, holding each node's total connection weight.

#### Step 1 — build the graph and its Laplacian

$L = D - W$ , where  $W_{ij}$  = similarity between nodes  $i$  and  $j$ , and  $D_{ii} = \sum_j W_{ij}$

Expand  $x^T D x - x^T W x$  term by term:  $\sum_i D_{ii} x_i^2 - \sum_i \sum_j W_{ij} x_i x_j$ . Expanding the right-hand side's  $(x_i - x_j)^2$  and using  $D_{ii} = \sum_j W_{ij}$  shows the two sides are identical term for term.

#### Step 2 — the key identity

$$x^T L x = \frac{1}{2} \sum_i \sum_j W_{ij} (x_i - x_j)^2$$

Every term in the sum is weighted by  $W_{ij}$ , so assigning very different real numbers to two strongly-similar (highly-weighted) nodes is expensive. Minimizing  $x^T L x$  therefore looks for a labeling that respects the graph's connectivity.

#### Step 3 — read what the identity means

*small  $x^T L x \Leftrightarrow$  strongly-connected nodes  $i, j$  (large  $W_{ij}$ ) are forced to have similar values  $x_i \approx x_j$*

The all-ones vector  $\mathbf{1}$  (every entry equal to 1) is always an eigenvector of  $L$  with eigenvalue 0 (every row of  $L$  sums to zero by construction) — a trivial solution that assigns every node the same value, excluded here by the orthogonality constraint. What remains is a continuous relaxation of finding the graph cut that severs the fewest, weakest edges.

#### Step 4 — the relaxed min-cut problem

minimize  $x^T L x$  subject to  $\|x\|=1$  and  $x \perp \mathbf{1}$

**By exactly the same Lagrange-multiplier argument as PCA (§12.3), the minimizer is the eigenvector of  $L$  with the smallest NON-ZERO eigenvalue — the Fiedler vector. Thresholding its entries (or feeding several low eigenvectors into ordinary k-means) recovers the graph partition. This is spectral clustering.**

**WORTH KNOWING — A MODERN ECHO: GRAPH NEURAL NETWORKS**

Graph Neural Networks generalize this idea by *learning* a similarity-aware embedding instead of relying purely on the graph Laplacian's fixed eigenvectors — message-passing layers repeatedly mix each node's representation with its neighbors', which is conceptually a learned, nonlinear stand-in for what the Laplacian's eigenvectors do in closed form. If spectral clustering is the linear-algebra version of “let the graph tell you the clusters,” GNNs are the deep-learning version of the same instinct.

## 13.7 Chapter 13 in one breath

**CHAPTER 13 IN ONE BREATH**

K-means divides data into  $k$  groups by alternately assigning points to the nearest center and re-centering to the mean — provably coordinate descent on the inertia objective, which always converges but only to a local optimum, since the assignment step is a non-convex, combinatorial choice. Its three biting points: sensitivity to initialization (fixed by k-means++), having to pre-choose  $k$  (elbow method or silhouette score), and an assumption of round, equal-sized clusters (fixed by GMM or DBSCAN for other shapes). Spectral clustering, which eigendecomposes a graph Laplacian by the very same Lagrangian argument that derived PCA, connects clustering directly to graph and relational data.

*End of Volume IV. Companion to Part IV of the ML/DL Handbook.*